RRF와 연결형 청킹을 통한 RAG 시스템의 효율성 최적화

김현수○, 하태빈, 박호진, 안영민

아이브릭스, AI-Tech
{hyunsookim, taebinha, hanwool, maniac}@i-bricks.co.kr

Optimizing the Efficiency of Retrieval-Augmented Generation through Reciprocal Rank Fusion and Linked Chunking

Hyun-soo Kim○, Tae-Bin Ha, Ho-Jin Park, Young-Min Ahn
I-BRICKS, AI-Tech

요 약

본 논문에서는 리랭커(Reranker)가 포함된 RAG(Retrieval-Augmented Generation) 시스템에서 발생하는 고비용의 추론 시간을 효과적으로 줄이는 것을 목표로 한다. 이를 위해 RRF(Reciprocal Rank Fusion) 기반의 융합 기법을 통하여 어휘적(Lexical) 결과와 의미적(Semantic) 결과를 적절하게 결합하여 리랭커를 대체할 수 있는지 가능성을 밝히고자 하였다. 또한, 일반적인 청킹(Chunking) 기법과 달리 하나의 문서로부터 파생된 청크들을 서로 연결하는 연결형 청킹(Linked Chunking) 구조를 통하여 청크들 간의 정보 손실을 줄이고 문맥 공유를 통해 검색 성능의 향상을 시킬 수 있음을 검증하였다. 최종적으로 실험을 통해 RRF와 연결형 청킹 기법이 기존 리랭커 기반 시스템과 대등한 성능을 보이면서 추론 속도를 효율적으로 단축시킬 수 있음을 확인하였다.

주제어: RAG, Reranking, RRF, Chunking

1. 서론

대형 언어 모델(Large Language Models, LLMs)은 자연어 처리 분야에서 비약적인 발전을 이끌었지만, 사전 학습된 내부 지식에만 의존하여 정보를 생성하는 구조적 특성으로 인해 환각(hallucination)이라는 근본적 한계를 지니고 있다. 이를 보완하기 위해 RAG(Retrieval-Augmented Generation)[1]가 활용되며, 이는 외부 지식원에서 사용자의 질의와 관련된 문서를 검색하여 프롬프트에 함께 제공함으로써 답변의 신뢰성과 정확도를 높이는 방식이다.

RAG의 성능은 근본적으로 검색 품질에 달려 있다. 정확하지 않거나 관련성이 낮은 정보가 검색될 경우, 이는 오히려 LLM의 답변 품질을 저하시키는 원인이 된다. 이러한 검색 품질 문제를 개선하기 위해 dense retrieval[2], adversarial retriever[3], hybrid query-based retrieval[4] 등 다양한 검색 기법이 연구되어 왔다. 최근 연구들은 단순히 정확한 문서를 찾는 것을 넘어, 프롬프트에 제공되는 정보들의 순서 또한 최종 답변의 품질에 큰 영향을 미치는 것으로 나타났다[5]. 이러한 연구에 따라서 검색된 문서들을 재정렬하는 리랭커(Reranker) 모델을 추가하는 것이 고성능 RAG 시스템의 핵심 기법으로 자리잡았다.

리랭커는 성능 향상 측면에서 매우 효과적이지만 질의와 검색된 문서들을 다시 한번 비교하며 복잡한 계산 과정을 거치게 된다. 이로 인해 필연적으로 느린 추론 속도와 높은 메모리 사용량으로 이어지고 실제 서비스에 적용하기 어려운 제약이 따른다.

본 연구에서는 이러한 한계를 극복하기 위한 대안으로 리랭커 없이 두 가지의 검색 결과를 융합하는 RRF(Reciprocal Rank Fusion)[6] 기법을 사용한다. 이 기법은 어휘(Lexical) 기반의 BM25 검색 결과와 의미(Semantic) 기반의 임베딩(Embedding) 모델 검색 결과를 RRF로 결합하여, 각 검색 방식의 장점을 상호 보완하면서 추론 시간과 자원 사용량을 효과적으로 줄일 수 있다.

또한, 긴 문서를 처리하는 과정에서 일반적으로 사용하는 청킹(Chunking) 기법은 문서를 일정한 크기로 분할하여 독립적인 단위로 처리되기 때문에 하나의 문서에서 생성된 청크들 간의 맥락이 단절될 수 있다. 본 논문에서는 이러한 정보 손실 문제를 완화하기 위해 같은 문서로부터 파생된 청크들 간의 관계를 유지하고 정보를 공유하는 연결형 청킹(Linked Chunking) 구조를 제안한다. 이를 통해 검색 대상 문서의 전체적인 맥락을 고려할 수 있고 더 정교한 검색이 가능하다.

본 연구는 실험을 통해 RRF 기반의 방식과 연결형 청킹 구조가 리랭커 모델과 유사한 성능을 달성하면서 효율성을 크게 향상시킬 수 있음을 보인다. 본 연구의 주요 기여는 다음과 같다.

1) 리랭커 모델 없이 BM25와 임베딩 검색 결과를 융합하는 RRF 기반 시스템을 제안하고, 실험을 통해 성능과 효율성 측면에서의 효과를 검증한다.

2) 문서 내 청크 간의 순서와 관계를 보존하는 연결형 청킹(Linked Chunking) 구조를 제안하고, 이를 통해 검색 성능을 향상시킬 수 있음을 보인다.

2. 관련 연구

2.1 RAG(Retrieval Augmented Generation)

Retrieval-Augmented Generation(RAG)은 대규모 언어 모델(LLM)의 한계를 보완하기 위해 외부 지식을 활용하는 대표적인 기법이다. RAG는 크게 검색 단계와 생성 단계로 구성되며, 먼저 대규모 문서 집합에서 질의와 관련된 정보를 검색한 뒤, 이를 LLM의 입력 프롬프트에 포함시켜 응답을 생성한다. 검색 단계에서는 BM25와 같이 단어 빈도 기반의 Sparse 검색기와 의미적 임베딩을 활용하는 Dense 검색기가 널리 사용된다. 또한 Cross Encoder 구조를 이용하여 검색된 문서를 재정렬하는 ReRanking 기법이 검색 성능을 크게 향상시켰다. 그러나 이러한 접근은 검색 외에 별도의 고비용 모델을 추가적으로 필요로 하므로 추론 시간과 메모리 사용량이 증가하는 단점이 있다. 본 연구에서는 이러한 계산적 비효율성을 완화하기 위해 RRF(Reciprocal Rank Fusion)[6]를 적용하여, RAG의 성능을 개선하고 계산 효율성을 크게 높이고자 한다.

2.2 RRF (Reciprocal Rank Fusion)

RRF(Reciprocal Rank Fusion)[6]은 정보 검색 분야에서 제안된 단순하면서도 강력한 Rank Fusion 기법으로, 서로 다른 검색 시스템에서 얻은 결과를 결합할 때 사용된다. RRF는 각 문서의 순위 정보를 역수로 변환해 가중합을 계산하므로, 개별 검색기가 산출하는 점수가 상이하거나 비교 불가능할 때도 안정적으로 결과를 통합할 수 있다.

$$f_{RRF}(q,d) = \frac{1}{\pi_{LEX}(q,d)+k} + \frac{1}{\pi_{SEM}(q,d)+k} \quad (1)$$

특히 어휘적(Lexical) 검색과 의미적(Semantic) 검색처럼 상보적인 특성을 지닌 방법들을 결합할 때 효과가 크다. 본 논문에서는 RRF가 Reranker에 비해 훨씬 적은 계산 비용으로 유사한 수준의 성능 달성을 보임과 동시에 LLM 기반 RAG 시스템에 실용적으로 적용될 수 있음을 보인다.

2.3 SRRF (Smooth Approximation of RRF)

SRRF(Smooth Approximation of RRF)[7]는 검색 결과의 순위(rank) 정보만 사용하는 기존 RRF의 한계를 보완하기 위해 제안되었다. SRRF의 가장 큰 특징은 sigmoid 함수를 이용하여 각 문서의 유사도 점수를 순위로 변환하는 점이다.

$$\sigma_\beta(x) = \frac{1}{1 + e^{-\beta x}} \quad (2)$$

수식 (2)의 경우 sigmoid 함수에 β 파라미터를 추가한 형태를 사용하고 있으며, 이 값이 커질 수록 그래프는 계단 함수(step function)와 유사해진다.

$$\tilde{\pi}(q,d) = 0.5 + \sum \sigma_\beta\left(f(q,d_j) - f(q,d_i)\right) \quad (3)$$

$$f_{SRRF}(q,d) = \frac{1}{\tilde{\pi}_{LEX}(q,d)+k} + \frac{1}{\tilde{\pi}_{SEM}(q,d)+k} \quad (4)$$

SRRF는 이 함수를 사용하여 특정 문서와 나머지 모든 문서의 유사도 점수를 비교하여 상대적인 순위를 계산한다. 최종적으로 수식 (4)와 같이 어휘적 순위 점수와 의미적 순위 점수를 결합하여 최종 점수를 도출하게 된다.

3.방법

본 연구는 RAG(Retrieval-Augmented Generation)[1] 시스템의 검색 정확도를 향상시키기 위해 긴 문서를 청킹(Chunking) 하여 작은 단위로 나눈 뒤, 이를 연결한 구조를 통해 최종 문서 벡터를 생성하는 방식을 적용하였다.

이를 위해 청킹 단계에서는 문장 분리, 고정 길이, 의미 기반 분할 방식을 적용하였으며, 문서 표현 단계에서는 최상위 선택과 임계값 기반 필터링(Threshold cutoff) 기법을 적용하여 최종 벡터를 생성하도록 하였다.

3.1 청킹(Chunking)

하나의 문서는 전체 텍스트를 이용하여 벡터화를 진행하기도 하지만, 일반적으로 긴 텍스트의 핵심 정보를 효과적으로 벡터화하기 위해 문서를 청크(Chunk) 단위로 분할하게 된다. 다양한 청킹 전략은 문서의 특성을 보존하는 방식에서 서로 다른 장점을 가지므로, 본 연구에서는 다음과 같은 세 가지 전략을 사용하여 문서의 청킹을 진행하고 성능 측정을 수행하였다.

3.1.1 문장 분리 기반 청킹

문서의 고유한 의미적 경계인 문장을 기준으로 분할하는 방식이다. 문장 단위는 그 자체로 의미가 완결된 단위이므로, 정보 손실 없이 의미적 경계를 보존할 수 있다. 본 연구에서는 NLTK(Natural Language Toolkit)[8, 9]를 이용하여 문장 분리 및 청킹을 진행하였다.

3.1.2 고정 길이 기반 청킹

문서를 지정된 토큰 길이에 따라 균일하게 분할하는 방식으로, 가장 일반적인 방식이다. 청크 간의 문맥이 단절되는 것을 방지하기 위해 인접한 청크 사이에 일정 길이의 텍스트를 중첩(Overlap)시키는 전략을 함께 사용하였다.

3.1.3 의미 기반 청킹

특정 단위나 길이가 아닌 의미적 유사도를 중심으로 청크를 구성하는 기법이다. 연속된 문장들의 벡터 간의 유사도를 측정하고, 유사도가 급격하게 변하는 지점을 청크의 경계로 설정한다. 이러한 특징으로 청크의 토큰 수는 균일하지 않을 수 있지만 의미적인 응집력이 높은 청크를 구성할 수 있다. 의미 기반 청킹은 Semantic Chunking 기법을 사용하여 유사 의미 단위끼리 묶어, 토

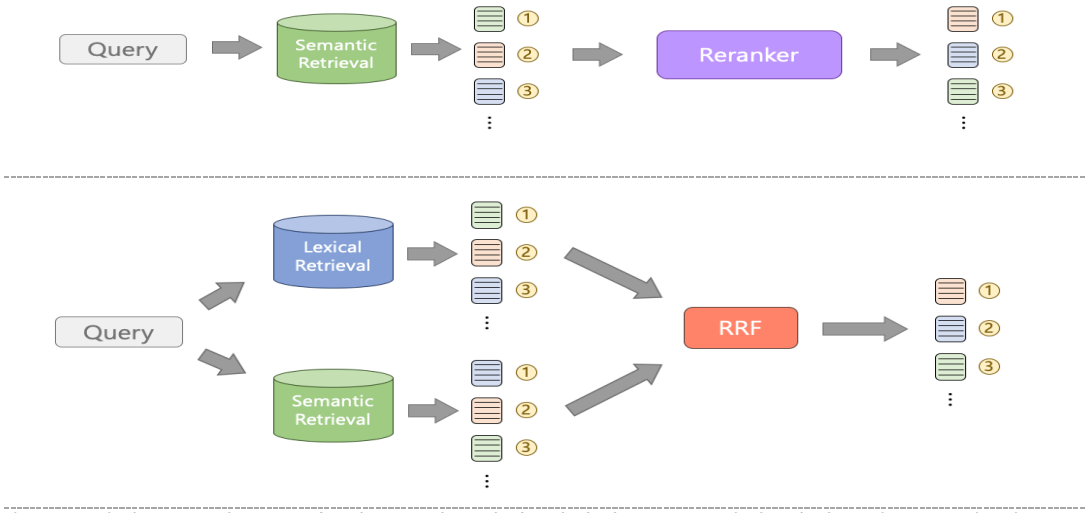

그림 1. 리랭커 기반 RAG 시스템(상)과 본 연구에서 제안하는 RRF 기반 하이브리드 검색 시스템(하) 비교

큰 수가 균등하지 않더라도 의미적 응집력이 높은 청크를 구성하였다.

3.2 연결형 청킹(Linked Chunking)

일반적인 청킹은 문서를 여러 개의 청크 단위로 분할하는 기법을 말하며, 이렇게 분할된 청크들은 독립적인 단위로 처리된다. 반면, 연결형 청킹(Linked Chunking)은 문서를 청크로 분할하는 점은 동일하지만 각 청크가 독립적으로 존재하는 것이 아니라 동일한 문서에서 파생된 청크 간의 순서와 관계를 유지하는 구조를 가진다.

본 연구에서 연결형 청킹을 적용한 목적은 크게 두 가지이다. 첫 번째, 문서를 단일 벡터로 표현할 때 발생할 수 있는 정보 손실을 줄이고, 청크 단위의 세분화된 분석을 통해 더 풍부한 정보를 획득하기 위함이다. 두 번째, 특정 청크의 관련성(relevance)을 평가할 때 해당 청크의 정보 뿐만 아니라 주변 청크들의 유사도를 현재 청크에 전파(propagate)하여 문맥을 반영한 최종 점수를 도출하기 위함이다.

$$\tilde{s}(i,j) = \gamma^{|i-j|} \times s_j \tag{5}$$

수식 (5)는 주변 청크 j의 유사도가 현재 청크 i에 미치는 영향을 계산하며, 거리가 멀어질수록 영향력이 감소하도록 γ계수를 적용한다. 여기서 s는 코사인 유사도(cosine similarity)를 의미한다.

$$f_{score}(i) = \alpha \times s_i + (1-\alpha) \times \sum_{j \neq i}^{k} \tilde{s}_j(i,j) \tag{6}$$

최종적으로 현재 청크 i의 고유 유사도 점수(s_i)와 주변 청크들로부터 전파된 유사도 점수의 합을 가중치(α)를 통해 결합하여 최종 점수를 계산한다.

3.3 청크 기반 문서 표현 (Chunk-based Document Representation)

이전 단계에서 생성된 다수의 청크 벡터들을 통합하여 문서 전체를 대표하는 단일 벡터를 생성하고, 질의(Query)에 대한 최종 점수를 계산하기 위해 아래와 같은 과정을 수행하였다.

3.3.1 최상위 청크 선택

문서에 속한 모든 청크 중에서 질의와 가장 높은 유사도를 갖는 단일 청크 벡터를 해당 문서의 대표 벡터로 선정하는 방식이다. 이는 전체 문서의 평균적 의미를 담기보다는 질의와 직접적으로 연관된 부분을 강조하여 검색 과정에서 불필요한 정보가 포함되는 것을 줄이고 응답 생성에 더 적합한 맥락을 제공하기 위함이다.

3.3.2 임계값 기준 필터링 (Threshold cutoff)

문서 내 모든 청크와 질의 간의 코사인 유사도를 계산한 뒤, 특정 임계값(Threshold) 이하인 관련성이 낮은 청크들은 결과에서 제외하는 방식이다. 이는 관련성이 낮은 청크들이 검색 결과의 잡음(noise)으로 작용할 수 있는 가능성을 방지하기 위함이다. 추가적으로 단순 합산으로 인해 일부 문서가 과도한 점수를 얻는 것을 막기 위하여 청크들의 유사도를 합산한 뒤 로그 함수를 적용해 정규화를 수행하였다.

4.실험 및 결과

4.1 데이터셋

실험에는 최근 LLM과 RAG 연구에서 많이 사용되는 BEIR[10] 데이터셋을 사용하였다. BEIR은 zero-shot 일반화 성능을 평가하는 IR 벤치마크이며 뉴스, 위키피디아, 금융, 의학 등 다양한 도메인을 포함한다. 본 논문에서는 아래의 표에 나열된 데이터에 대해 실험을 진행했다.

표 1. 데이터셋

데이터셋	쿼리(Query)	문서(Corpus)
Trec-covid	50	171K
NFCorpus	323	3.6K
Touche-2020	49	382K
Quora	10,000	523K
Scidocs	1,000	25K
FEVER	6,666	5.42M
Climate-FEVER	1,535	5.42M
Scifact	300	5K
HotpotQA	7,405	5.23M
ArguAna	1,406	8.67K

4.2 베이스라인 (Baseline)

성능 비교 실험을 위해 본 연구에서는 여러 가지 베이스라인 모델을 설정하였다. 전통적인 BM25 검색기를 사용하여 질의와 문서 간의 단어 빈도 기반 점수를 계산하는 방법과 임베딩 모델을 사용하는 의미 기반 검색을 베이스라인으로 두어 질의와 문서 임베딩 벡터 간의 코사인 유사도를 구하여 검색 성능을 평가하였다. 또한 검색 속도 비교를 위해 그림 1과 같이 리랭커 모델을 적용한 RAG 시스템을 추가 베이스라인으로 설정하였다. 리랭커를 이용한 베이스라인은 BM25와 임베딩 모델 두 개의 결과를 각각 재정렬하는 방식으로 구성하였다. 실험에 사용한 임베딩과 리랭커 모델은 각각 *BAAI/bge-m3*와 *BAAI/bge-reranker-v2-m3*를 사용하였다.

4.3 평가 지표

본 연구의 검색 성능을 평가하기 위해 NDCG(Normalized Discounted Cumulative Gain)[11]를 평가 지표로 사용한다. Recall, Precision, MRR 및 MAP 등의 다양한 평가 지표들이 사용되고 있지만 이러한 지표들은 순위를 반영하지 않거나 관련성 여부를 이진적(0 또는 1)으로만 판단하는 한계가 있다. 반면, NDCG는 문서의 순위와 관련성 점수를 모두 고려하므로 검색 성능을 더 정교하게 측정할 수 있다.

$$DCG_D(f, S_n) = \sum_{r=1}^{n} y_{(r)}^{f} D(r). \quad (7)$$

$$NDCG_D(f, S_n) = \frac{DCG_D(f, S_n)}{IDCG_D(S_n)} \quad (8)$$

DCG는 수식 (7)과 같이 계산된다. 여기서 $y_{(r)}^{f}$값은 순위 r에 위치한 문서의 관련성 점수를 의미한다. $D(r)$은 Discount function으로 $\frac{1}{\log(1+r)}$을 의미하며, n값이 커질수록 기여도를 점점 감소시키는 역할을 한다. IDCG는 이상적인 순위 즉, 관련성이 높은 문서부터 정렬되어 있을 경우 얻을 수 있는 DCG의 최대값($max_{y'} \sum_{r=1}^{n} y_{(r)}^{y'} D(r)$)으로 이해할 수 있다. 최종적으로 NDCG는 수식 (8)과 같이 DCG값을 IDCG로 나누어 계산할 수 있으며 1에 가까울수록 이상적인 성능에 가까운것을 의미한다.

4.4 실험 결과 및 분석

4.4.1 청킹 성능 분석

효과적인 청킹 전략을 파악하기 위해 3.1절에서 소개한 고정 길이 기반, 문장 분리 기반, 의미 기반 청킹 방식에 대해서 성능을 비교 분석하였다. 실험에 사용한 데이터는 ArguAna, NFcorpus, Scifact 데이터셋을 사용하였으며 그 결과는 아래 표와 같다.

표 2. 청킹 방식에 따른 성능 비교(nDCG@10)

데이터셋	고정 길이	문장	의미
ArguAna	**0.523**	0.467	0.521
NFcorpus	**0.362**	0.360	0.353
Scifact	**0.721**	0.688	0.706

표 2에서 보이는 것 처럼 모든 데이터셋에서 고정 길이 기반 청킹 방식이 가장 우수한 성능을 보였다. 이는 청크가 적절한 단위로 구성될 때, 질의와 관련된 풍부한 문맥이 하나의 청크에 포함될 가능성이 높음을 보여준다. 의미 기반 청킹은 고정 길이와 문장 단위 청킹과 비슷한 성능을 내지만 선행 연구[12]에서와 마찬가지로 추가 계산 비용이 필요하기 때문에 이 후의 실험에서는 고정 길이 기반 청킹을 기본 방식으로 채택했다. 실험에서 사용한 고정 길이는 500 토큰, 중첩(오버랩)은 50으로 진행하였다.

4.4.2 청크 기반 문서 표현 성능 분석

분할된 청크들을 통합하여 문서 전체를 표현하는 방식의 효과를 분석한다. 문서를 표현하는 방식은 3.2절에서 제시한 최상위 청크 선택 방식과 임계값 기준 필터링 방식의 성능을 비교했으며 결과는 다음 표와 같다.

표 3. 문서 표현 성능 비교

데이터셋	최상위 청크	임계값
ArguAna	**0.52968**	0.52087
NFcorpus	**0.35683**	0.3441
Scifact	**0.69779**	0.69556

표 4. 각 모델의 검색 성능을 nDCG@10 기준으로 비교. BM25(Lexical)와 Embedding(Semantic) 검색 결과를 RRF/SRRF로 융합한 방식과 본 논문에서 제안하는 연결형 청킹(Linked Chunking)을 적용한 방식의 성능을 함께 제시한다. 각 데이터셋에서 가장 높은 점수를 **굵게** 표시하였다.($\boldsymbol{RNK}$는 Reranker, $\boldsymbol{DIFF}$는 RRF/SRRF가 기록한 최고 성능과 Reranker의 최고 성능 간의 차이를 나타내며, 아랫 첨자 $\boldsymbol{B,E,C,L}$는 각각 BM25, Embedding, 기본 청킹, 연결형 청킹을 의미한다.)

$Dataset/Model$	RNK_B	RNK_E	$RRF_{(E,B)}$	$SRRF_{(E,B)}$	$RRF_{(C,B)}$	$SRRF_{(C,B)}$	$RRF_{(L,B)}$	$SRRF_{(L,B)}$	$DIFF$
Trec-Covid	0.815	0.814	0.798	0.771	0.784	0.759	**0.818**	0.780	**+0.003**
NFCorpus	0.355	0.331	0.365	**0.369**	0.360	0.363	0.359	0.362	**+0.014**
Touche-2020	0.351	0.356	0.355	0.356	0.398	0.393	0.394	**0.410**	**+0.054**
Quora	0.889	**0.891**	0.877	0.885	0.877	0.885	0.876	0.886	**−0.005**
Scidocs	0.168	0.174	0.183	**0.187**	0.178	0.179	0.178	0.180	**+0.013**
Fever	0.876	**0.896**	0.793	0.819	0.803	0.812	0.817	0.826	**−0.070**
Climate-Fever	0.316	**0.367**	0.290	0.292	0.272	0.262	0.277	0.269	**−0.075**
Scifact	**0.732**	0.724	0.703	0.703	0.716	0.718	0.717	0.721	**−0.010**
HotpotQA	0.777	**0.816**	0.717	0.720	0.717	0.718	0.716	0.709	**−0.096**
ArguAna	0.480	0.432	0.534	**0.537**	0.517	0.523	0.497	0.498	**+0.057**

표 5. 모델별 평균 추론 시간(초) 비교. Reranker 모델의 소요 시간은 1차 검색(BM25 또는 Embedding)과 리랭킹(Reranking) 시간을 합산한 값이다. RRF 및 SRRF 모델은 BM25와 Embedding 검색을 수행한 시간과, 두 결과를 융합하는 계산 시간을 포함한다.($\boldsymbol{RNK}$는 Reranker, 아랫 첨자 $\boldsymbol{B,E,C,L}$는 각각 BM25, Embedding, 기본 청킹, 연결형 청킹을 의미한다.)

	RNK_B	RNK_E	$RRF_{(E,B)}$	$SRRF_{(E,B)}$	$RRF_{(C,B)}$	$SRRF_{(C,B)}$	$RRF_{(L,B)}$	$SRRF_{(L,B)}$
소요 시간(초)	6476.47	8466.14	394.36	453.79	655.47	714.95	655.42	726.51

Arguana 데이터셋과 Scifact 데이터셋에서는 최상위 청크 방식과 임계값 방식이 거의 동일한 성능을 보였다. 이는 질의와 가장 유사한 청크를 선택하거나 일정 유사도 이상의 청크만으로 문서를 표현하는 방법 모두 효과적임을 보여준다. 그러나 NFcorpus 데이터셋에서는 성능 차이가 다소 발생하였다. 이는 임계값 방식은 임계치를 넘는 청크 중에도 실제로는 중요하지 않은 청크가 포함되는 경우와 임계치 미달인 경우에도 중요한 정보가 제외될 가능성이 있어 성능이 다소 제한될 수 있음을 보여준다. 즉, 데이터셋 특성에 따라 최적의 문서 표현 방식은 달라질 수 있음을 확인할 수 있다.

4.4.3 연결형 청킹 및 RRF/SRRF 성능 분석

이번 절에서는 본 연구의 핵심 목표인 리랭커 모델과 제안하는 RRF/SRRF 방식의 성능 및 효율성을 종합적으로 비교 분석한다. 앞선 실험 결과를 바탕으로 고정 길이 청킹과 최상위 청크 선택 방식을 기본으로 적용한다. 또한, RRF와 SRRF의 k는 60, β는 40으로 설정하고 연결형 청킹의 f_{score}에 대한 α는 0.9, γ는 0.7로 고정하여 실험을 진행하였다.

표 4에서는 데이터셋 별 다양한 모델들의 대한 nDCG@10의 값을 보여준다. Trec-covid와 Touche-2020 데이터셋에서는 연결형 청킹의 성능이 리랭커를 포함한 모든 모델 중 가장 뛰어난 것을 확인할 수 있다. 또한 NFCorpus, Scidocs, ArguAna에서는 RRF, SRRF를 적용한 것이 고비용의 리랭커 모델보다 더 높은 성능을 기록했다. 리랭커 모델이 가장 높은 성능을 보인 Quora, Scifact 데이터셋에서도 제안하는 연결형 청킹 모델이 리랭커 점수에 매우 근접한 성능을 달성했다.

표 5는 모델을 수행하는데 소요되는 평균 소요 시간을 나타낸다. 리랭커는 모든 모델들 중 가장 긴 추론 시간을 요구하지만 RRF 및 SRRF를 적용한 모델은 매우 적은 추론 시간을 필요로 한다. 또한, RRF, SRRF는 모델을 사용하지 않기 때문에 컴퓨팅 자원 사용량도 줄일 수 있다.

결론적으로, 제안하는 RRF/SRRF 융합 방식과 연결형 청킹은 리랭커 모델과 대등하거나 또는 더 나은 성능을 달성하면서 추론 시간을 줄일 수 있는 것을 확인할 수 있으며 리랭커의 대안이 될 수 있다.

5. 결론

본 연구에서는 RAG 시스템의 검색 속도 저하를 일으키는 리랭커 모델을 대체하기 위해 RRF/SRRF 기반의 하이브리드 검색 시스템을 제안한다. 또한 검색 정확도를 높이기 위한 연결형 청킹 구조를 제시하였다. 이를 통한 기여점은 아래와 같다.

1. 다양한 실험을 통해 RRF/SRRF를 이용한 하이브리드 기법이 리랭커 모델을 대체할 수 있음을 보였다.

2. 문서 내 청크 간의 순서와 문맥 관계를 보존하는 연결형 청킹을 제안하고, 실험을 통해 성능이 향상됨을 보였다.

향후 연구로 연결형 청킹의 수식을 고도화 하고, 적절한 하이퍼 파라미터를 탐색할 예정이다. 이를 통해 범용적으로 사용할 수 있는 문서 구조를 확립하고 개선 정도를 측정할 예정이다.

참고문헌

[1] Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in neural information processing systems 33, 9459-9474, 2020.
[2] Karpukhin, Vladimir, et al. "Dense Passage Retrieval for Open-Domain Question Answering." EMNLP (1), 6769-6781, 2020.
[3] Zhang, Hang, et al. "Adversarial retriever-ranker for dense text retrieval.", arXiv:2110.03611, 2021.
[4] Sawarkar, Kunal, et al. "Blended rag: Improving rag (retriever-augmented generation) accuracy with semantic search and hybrid query-based retrievers." 2024 IEEE 7th international conference on multimedia information processing and retrieval (MIPR), 155-161, 2024
[5] Liu. Nelson F., et al "Lost in the middle: How language models use long contexts." arXiv preprint arXiv:2307.03172, 2023
[6] Cormack, Gordon V., et al. "Reciprocal rank fusion outperforms condorcet and individual rank learning methods." Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval, 758-759, 2009.
[7] Bruch, Sebastian, et al. "An analysis of fusion functions for hybrid retrieval." ACM Transactions on Information Systems 42.1, 1-35, 2023
[8] Bird, Steven. "NLTK: the natural language toolkit." Proceedings of the COLING/ACL 2006 interactive presentation sessions. 2006
[9] Kiss, Tibor, and Jan Strunk. "Unsupervised multilingual sentence boundary detection." Computational linguistics 32.4:485-525, 2006
[10] Thakur, Nandan, et al. "Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models.", arXiv:2104.08663, 2021.
[11] Wang, Yining, et al. "A theoretical analysis of NDCG type ranking measures." Conference on learning theory. PMLR, 2013.
[12] Qu, Renyi, et al. "Is semantic chunking worth the computational cost?.", arXiv:2410.13070, 2024.